

윈도우 프로그래밍

2. C# 프로그래밍 첫걸음

2026. 3. 13
심미나



목 차

- I. 플랫폼과 프로그래밍 언어
- II. 라이브러리와 프레임워크
- III. C#으로 할 수 있는 일
- IV. 실습 환경 구축
- V. 실습 및 과제

I. 플랫폼과 프로그래밍 언어

플랫폼과 프로그래밍 언어



C#의 특징

- 형식 안정적(Type-Safe)인 객체 지향(Object-Oriented) 언어
- 기존 프로그래밍 언어의 생산성을 개선하여 성능이 높음
- 윈도우, 맥, 리눅스, 안드로이드, 아이폰 등의 다양한 운영체제나 플랫폼에서 동작
 - 윈도우에서 동작하는 닷넷 플랫폼과 대부분의 운영체제에서 동작하는 모노 플랫폼에서 작동하는 프로그램을 만들 수 있음

플랫폼과 프로그래밍 언어



플랫폼과 소프트웨어 플랫폼

- 플랫폼(Platform)

- 소프트웨어 응용 프로그램의 실행에 사용되는 하드웨어와 소프트웨어의 집합
 - 윈도우, 맥, 안드로이드(리눅스), 아이폰(iOS) 등은 해당 운영체제 상에서 다양한 프로그램을 실행 가능한 플랫폼

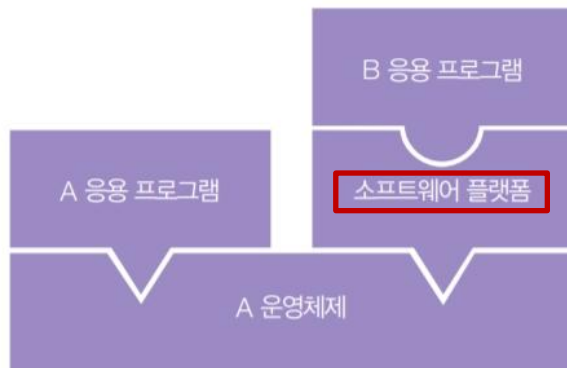


플랫폼과 프로그래밍 언어

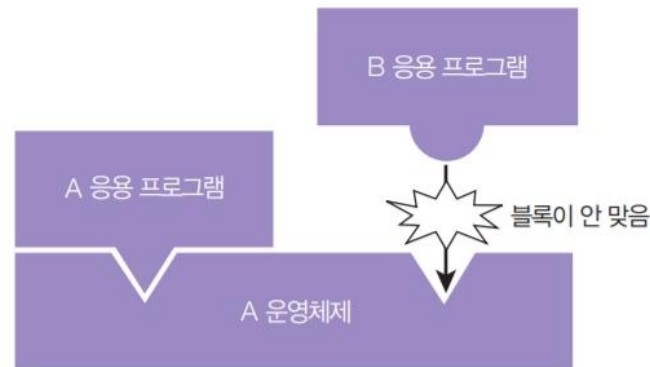


소프트웨어 플랫폼

- 소프트웨어 플랫폼은 그림과 같이 중간 레고 블록 역할을 함
 - 예 : 자바 가상 머신, 액션스크립트, 닷넷 플랫폼



(b) 소프트웨어 플랫폼이 있는 경우: B 운영체제용으로 만들어진 B 응용 프로그램을 A 운영체제에서 실행 가능



(a) 소프트웨어 플랫폼이 없는 경우: B 운영체제용으로 만들어진 B 응용 프로그램을 A 운영체제에서 실행 불가

플랫폼과 프로그래밍 언어



닷넷 플랫폼

- 닷넷 플랫폼은 MS사가 만든 중간 레고 블록
 - 초기에는 윈도우 운영체제에 종속된 형태로 디자인됨
 - 현재는 MS사가 활용하는 모든 프로그래밍 언어(20개 이상)를 닷넷 플랫폼과 연결
: 닷넷 플랫폼이 ‘공통 언어 인프라’(CLI, Common Language Infrastructure)를 구현한
‘공통 언어 런타임’(CLR, Common Language Runtime)이기 때문



플랫폼과 프로그래밍 언어



MS사의 닷넷 프레임워크

- 닷넷 플랫폼(CLR)과 클래스 라이브러리를 함께 제공
- 윈도우에서 동작하는 다양한 응용 프로그램 개발 가능
 - 응용 프로그램 프레임워크: 윈도우 폼, WPF, 윈도우 유니버설 응용 프로그램
 - 웹 응용 프로그램 프레임워크: 실버라이트
 - 웹 서버 프레임워크: ASP.NET, ASP.NET MVC
 - 웹 서비스 프레임워크: WCF

플랫폼과 프로그래밍 언어



닷넷 프레임워크



- 문라이트 프로젝트 (모노 프로젝트의 일환)
 - 닷넷 프레임워크를 리눅스에서도 동작하도록 구현해서 C#이 리눅스에서도 동작함
- 2014년, .NET을 오픈소스화하고 .NET Core 이름으로 모노를 공식지원
- 2020년, .NET Core(모노) 플랫폼이 정식 플랫폼이 됨
 - 대부분의 라이브러리가 대부분의 운영체제에서 동작할 수 있게 됨



플랫폼과 프로그래밍 언어



C#의 기타 활용

- 게임 프레임워크(게임 엔진): 유니티
- 모바일 응용 프로그램 프레임워크(안드로이드와 아이폰): Xamarin
- 머신러닝과 딥러닝: ML.NET

II. 라이브러리와 프레임워크

라이브러리와 프레임워크



라이브러리

- 코드를 쉽게 사용할 수 있게 미리 만들어준 코드
 - 프로그램 소프트웨어를 만들 때 사용하는 클래스 또는 서브루틴 집합
- 라이브러리는 자체 실행 안되며 사용자 코드를 통해 호출하여 사용
 - 즉, 개발자가 사용해줘야만 하며, 스스로는 아무것도 하지 못함

사용자 코드

```
int Main() {
```

```
    Console.WriteLine("출력");
```

```
    Math.Abs(-273);
```

```
    new Thread();
```

```
}
```

라이브러리

```
void WriteLine() {...}
```

```
int Abs() {...}
```

```
public Thread() {...}
```

라이브러리와 프레임워크



프레임워크

- 제어 역전(IoC, Inversion of Control)이 있는 대규모의 라이브러리
 - 제어역전 : 원래 개발자가 제어하던 코드를 프레임워크가 제어하는 것
- 프로그램의 초기화부터 종료까지의 흐름을 직접 관리
 - 기본 틀(Framework)을 모두 제공해주어 개발자는 개발에 집중 가능
 - 현재는 "대규모의 라이브러리 = 프레임워크"로 많이 혼용
 - * 닷넷 프레임워크는 닷넷 플랫폼과 클래스 라이브러리가 합쳐진 하나의 제품 이름

사용자 코드

```
void Start() {...}  
void Update() {...}  
void End() {...}
```

프레임워크

```
int Main() {  
    Start();  
    while(!isEnd) { update(); }  
    End();  
}
```

III. C#으로 할 수 있는 일

C#으로 할 수 있는 일



GUI(Graphical User Interface) 개발

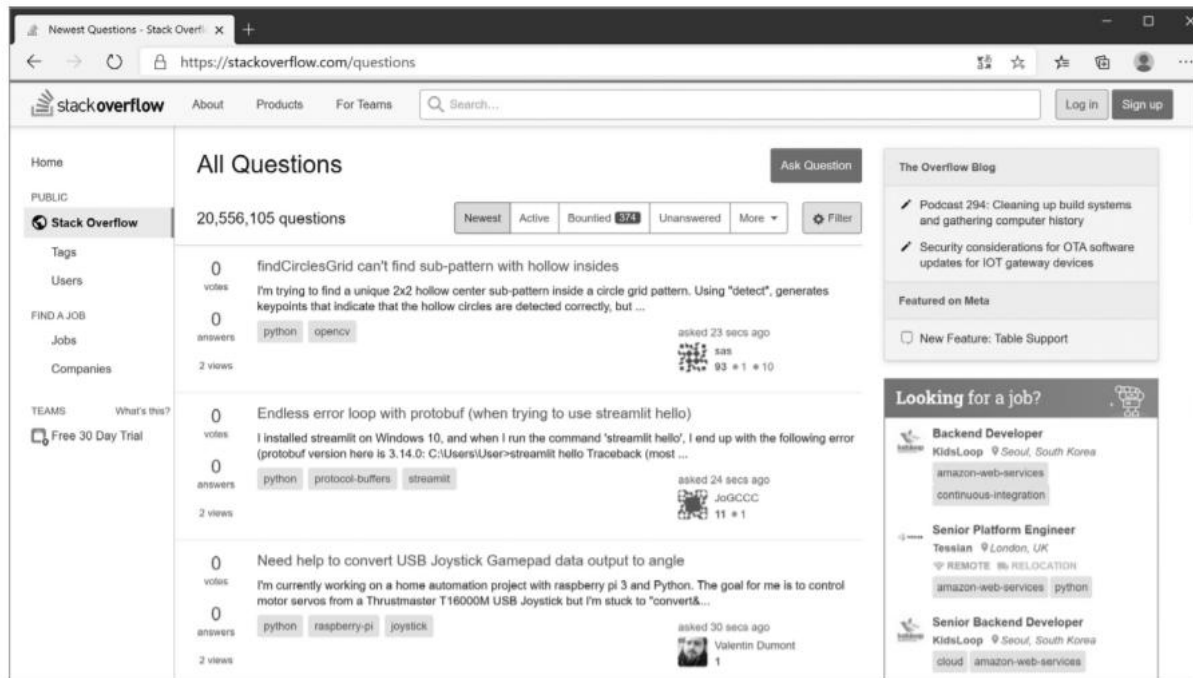
- 윈도우에서 작동하는 GUI 프로그램 개발 위한 윈도우 폼과 WPF 제공
- 윈도우 폼(Windows Form)
 - C#의 가장 기초 프레임워크로, C++를 사용한 윈도우 개발을 C#으로 옮겨 놓은 형태
 - 개발자가 폼 디자이너 이용해 도구 상자에서 버튼이나 콤보 박스 등의 컨트롤을 윈도우에 배치할 때마다 폼 디자이너가 프로그램의 UI를 표시하면서 C# 코드 자동 생성
- WPF(Windows Presentation Foundation)
 - 현대적인 개발 패턴 중에 MVC 패턴과 MVVM 패턴을 적용해 개발 생산성을 향상시킨 프레임워크
 - DirectX 등의 기능도 추가로 내장하여 3D 그래픽까지 자체적으로 처리 가능

C#으로 할 수 있는 일



웹 개발

- C#이 가장 대표적으로 사용되는 부분
- MS사는 2개 프레임워크(ASP.NET 프레임워크와 ASP.NET MVC)를 지원
 - * ASP.NET MVC로 개발된 스택오버플로

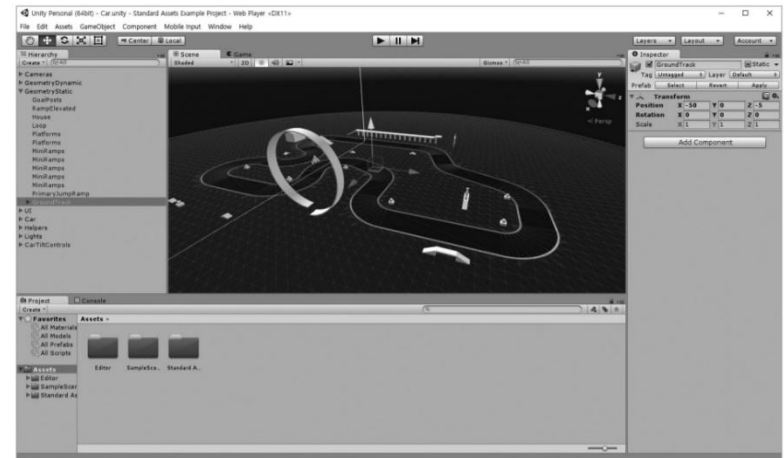


C#으로 할 수 있는 일



게임 개발

- C#을 이용해 게임 클라이언트와 게임 서버 개발 가능
- 게임 클라이언트 개발
 - 유니티 엔진 개발로 C#으로 게임 클라이언트 개발 활성화
 - 유니티는 모노 플랫폼을 사용해 다양한 플랫폼에서 작동하는 게임 개발 가능
- 게임 서버 개발
 - 간단한 모바일 게임 서버는 웹 서버처럼 개발
 - 대규모 MMORPG 게임 서버도 개발

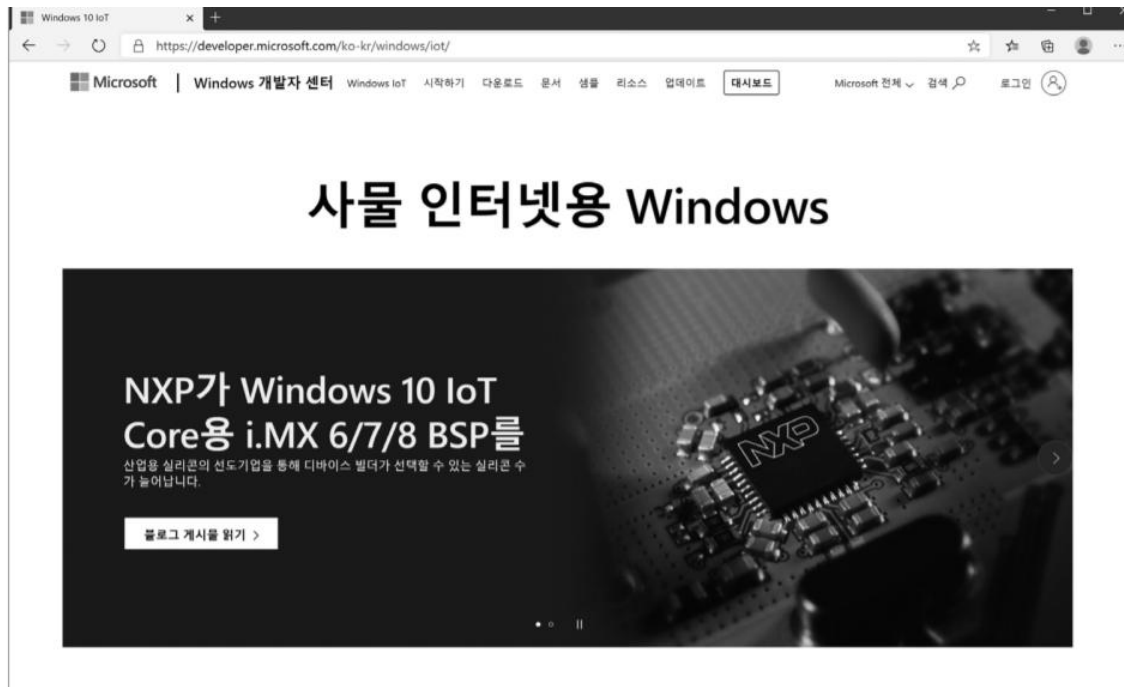


C#으로 할 수 있는 일



IoT(사물 인터넷^{Internet of Things: IoT}) 개발

- C#을 사용해 다양한 IoT 개발 가능
 - IoT는 다양한 사물에 센서와 통신 기능을 내장하고 인터넷과 연결하는 기술
 - * *Window10 IoT 코어, Window10 IoT Enterprise, Window10 IoT Desktop*



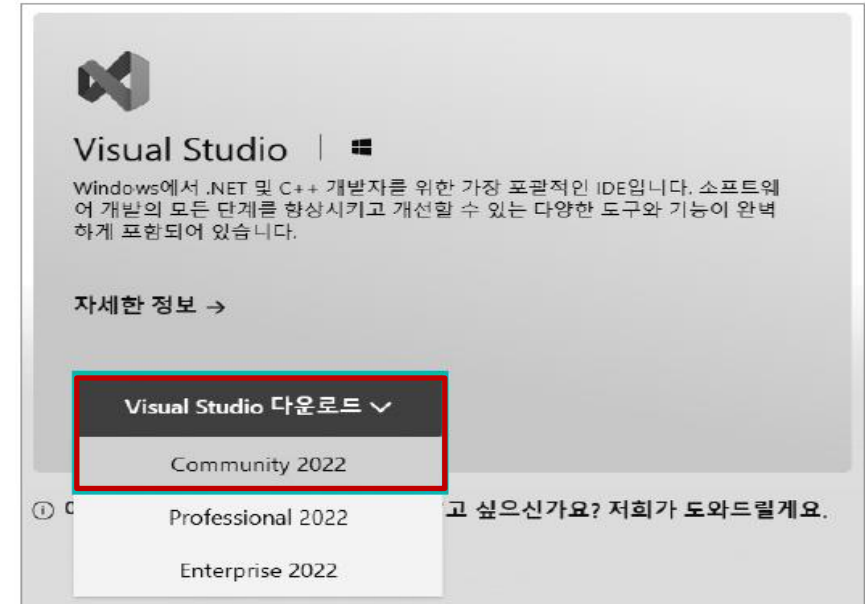
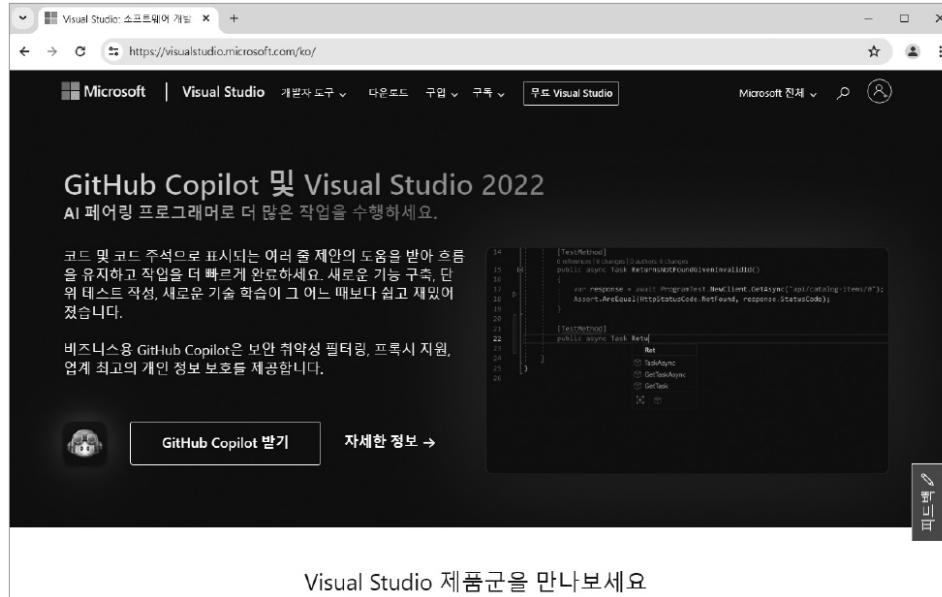
IV. 실습 환경 구축

실습 환경 구축



Visual Studio 2022 설치

- <https://visualstudio.microsoft.com/ko>에서 ‘Visual Studio 다운로드’
 - 버전 목록에서 ‘Community 2022’ 선택
 - (기타 링크) https://aka.ms/vs/17/release/vs_community.exe 다운로드

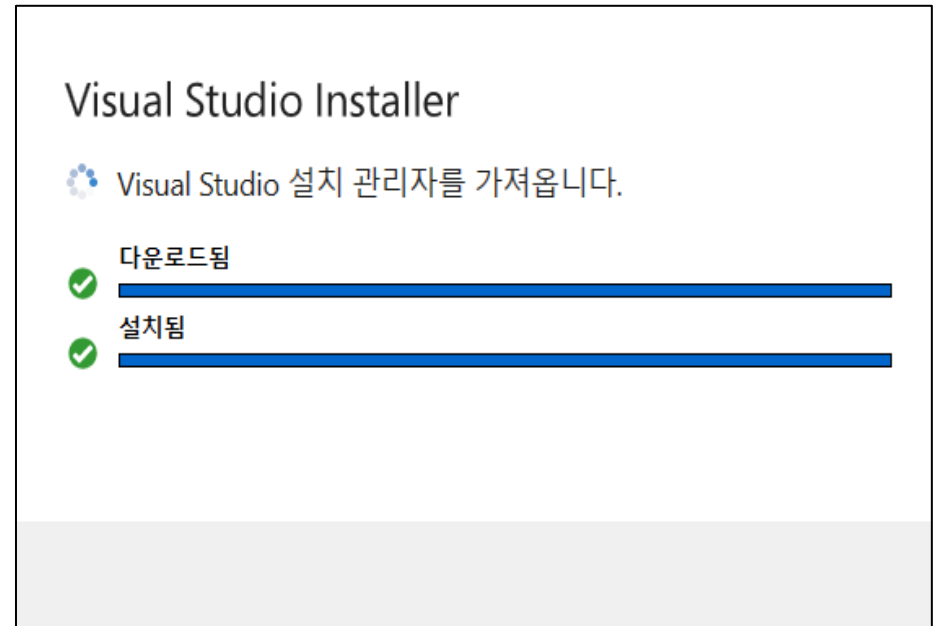
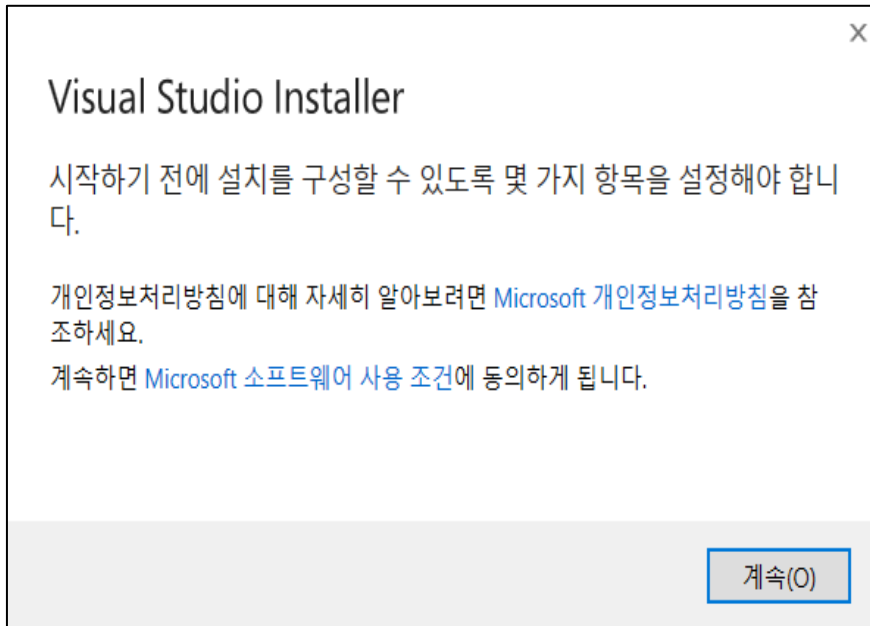


실습 환경 구축



Visual Studio 2022 설치

- 다운로드한 파일(VisualStudioSetup.exe) 실행, 설치관리자가 설치됨

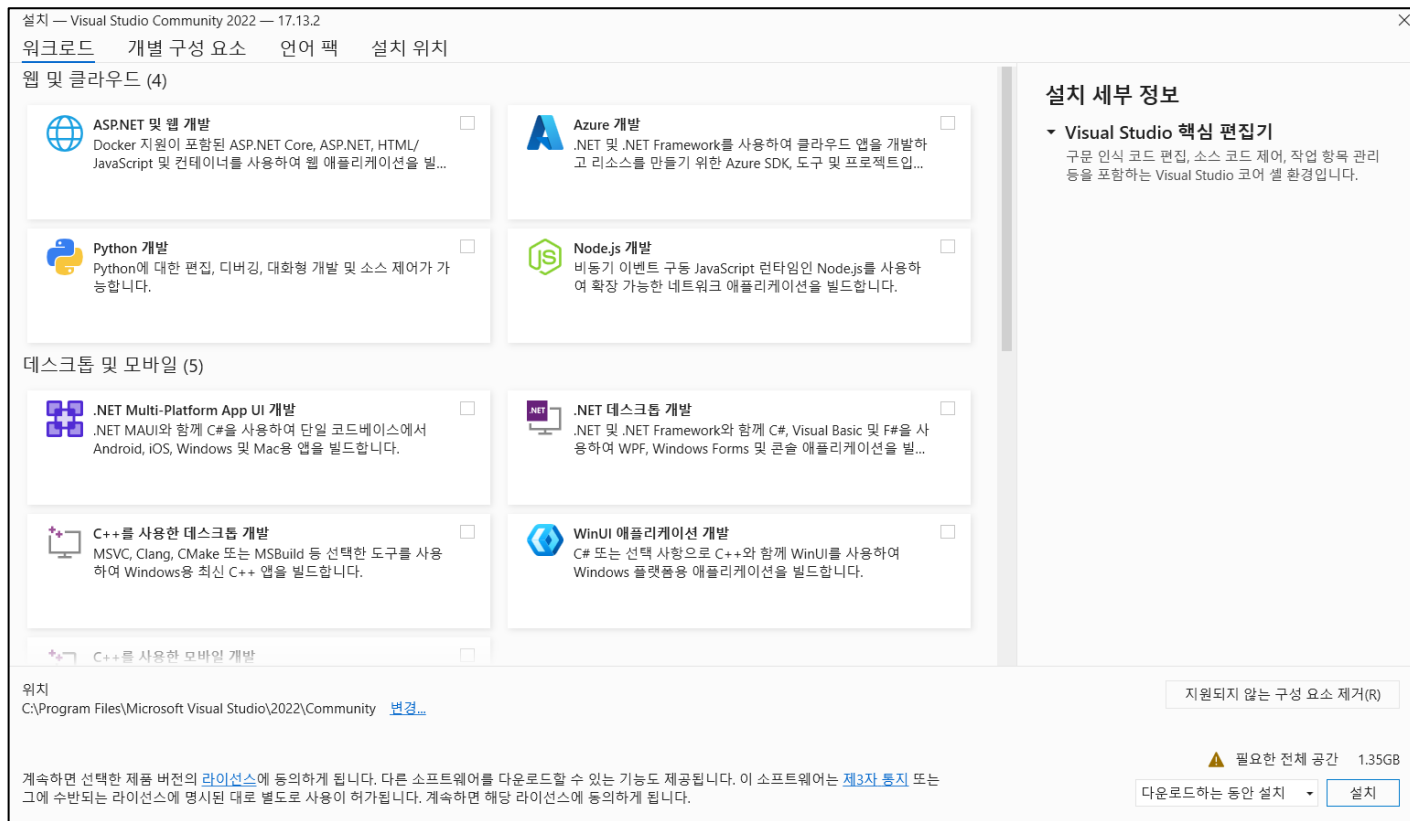


실습 환경 구축



Visual Studio 2022 설치

- 설치관리자가 설치되면 다음과 같은 화면이 열림

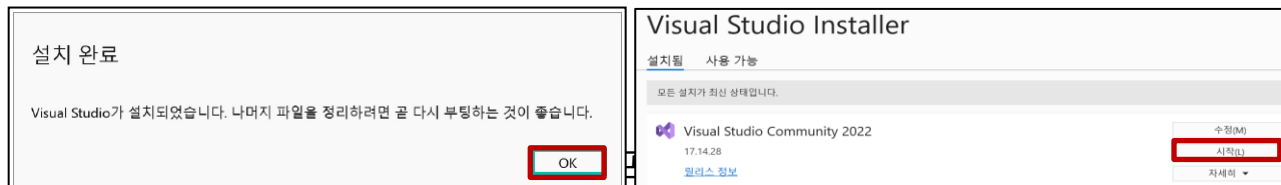


실습 환경 구축



Visual Studio 2022 설치

- [데스크톱 및 모바일]-[.NET 데스크톱 개발]에 체크하고, 우측 하단 [설치]
- 설치가 모두 완료되면, 다음과 같은 [설치 완료] 대화상자가 표시됨

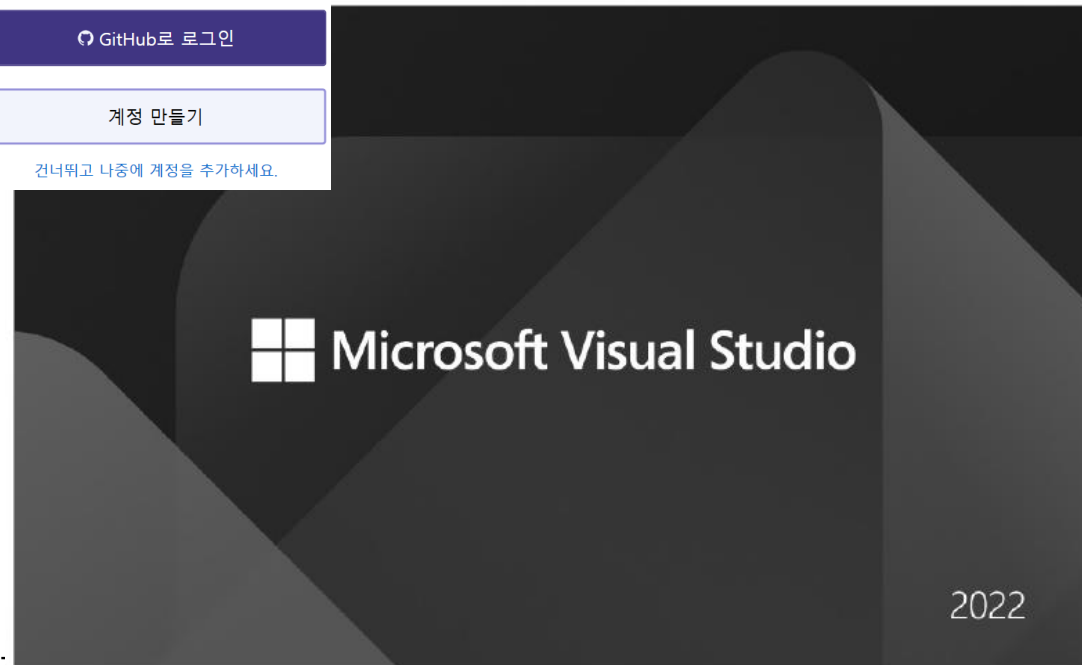
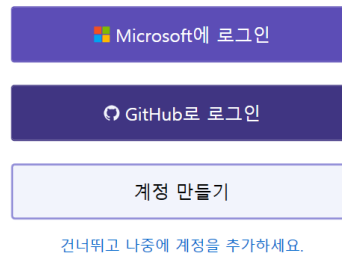
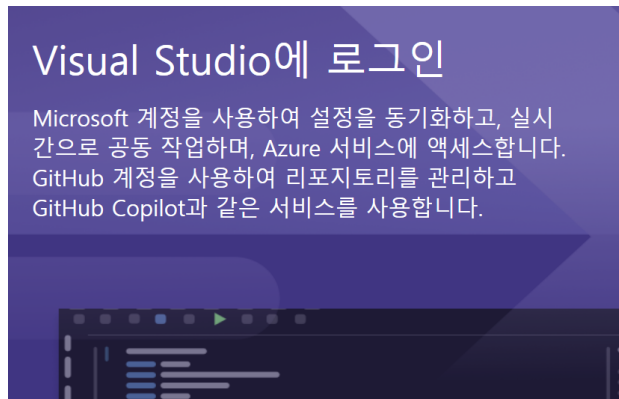


실습 환경 구축



Visual Studio 2022 첫 실행

- Visual Studio 2022를 첫 실행하면 다음과 같은 스플래시 스크린이 표시

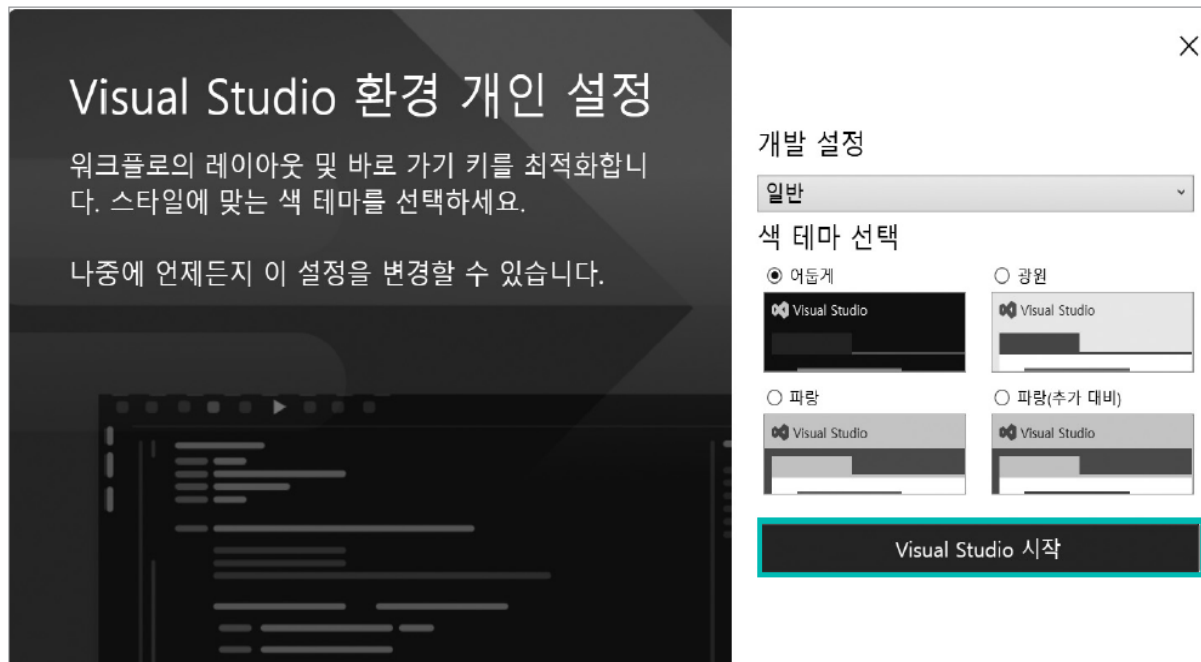


실습 환경 구축



Visual Studio 2022 첫 실행

- [Visual Studio 환경 개인 설정] 대화상자에서 원하는 색 테마를 선택한 뒤 [Visual Studio 시작]을 클릭

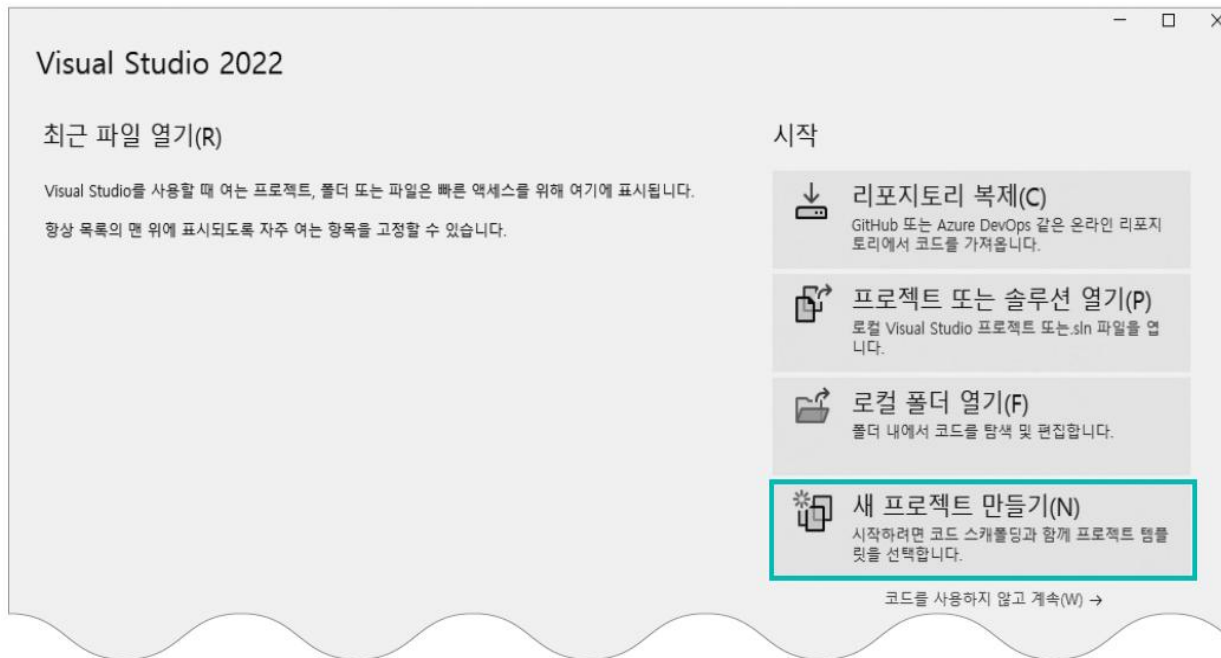




프로그램 생성

- 프로젝트 생성

- Visual Studio 2022를 실행하고 [새 프로젝트 만들기] 버튼을 클릭



실습 환경 구축



프로그램 생성

- 프로젝트 생성
 - [새 프로젝트 만들기] 대화상자에서 C# 콘솔 앱을 선택





프로그램 생성

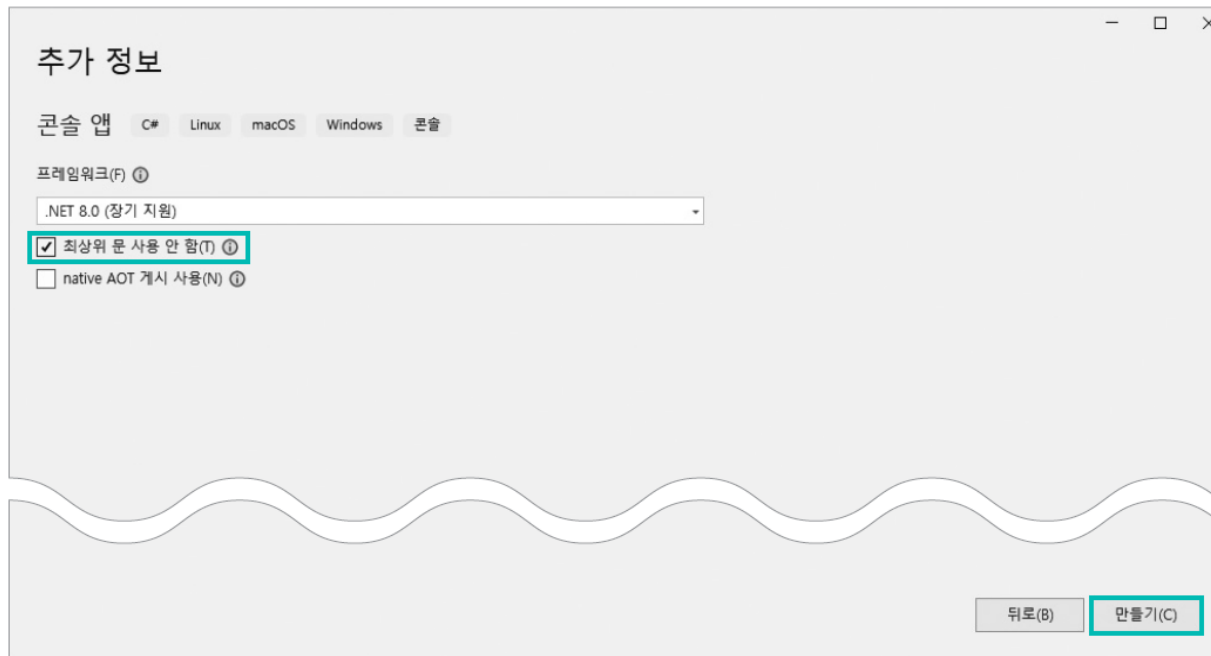
- 프로젝트 생성

- [새 프로젝트 구성] 대화상자가 표시되면, [프로젝트 이름]을 "FirstProgram"으로 변경



프로그램 생성

- 프로젝트 생성
 - [추가 정보] 대화상자가 표시되면, 반드시 [최상위 문 사용 안 함]에 체크



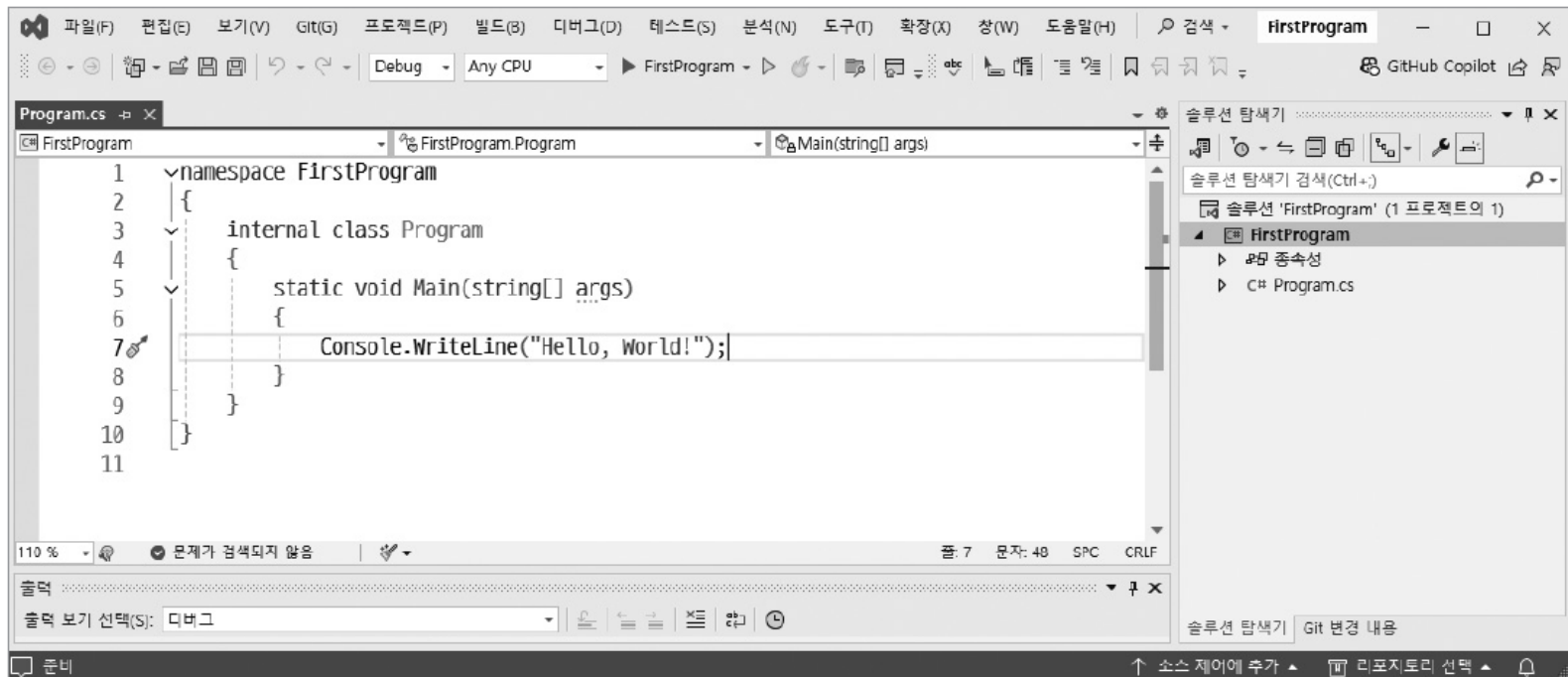
실습 환경 구축



프로그램 생성

- 프로젝트 생성

- 프로젝트를 만들면 다음과 같은 화면이 표시되고, 오른쪽 [솔루션 탐색기] 패널을 보면 프로젝트의 구조를 확인할 수 있음





프로그램 생성

- 프로젝트 생성
 - 프로그램의 중심이 되는 Program.cs 파일을 열어보면 다음과 같이 작성되어 있음

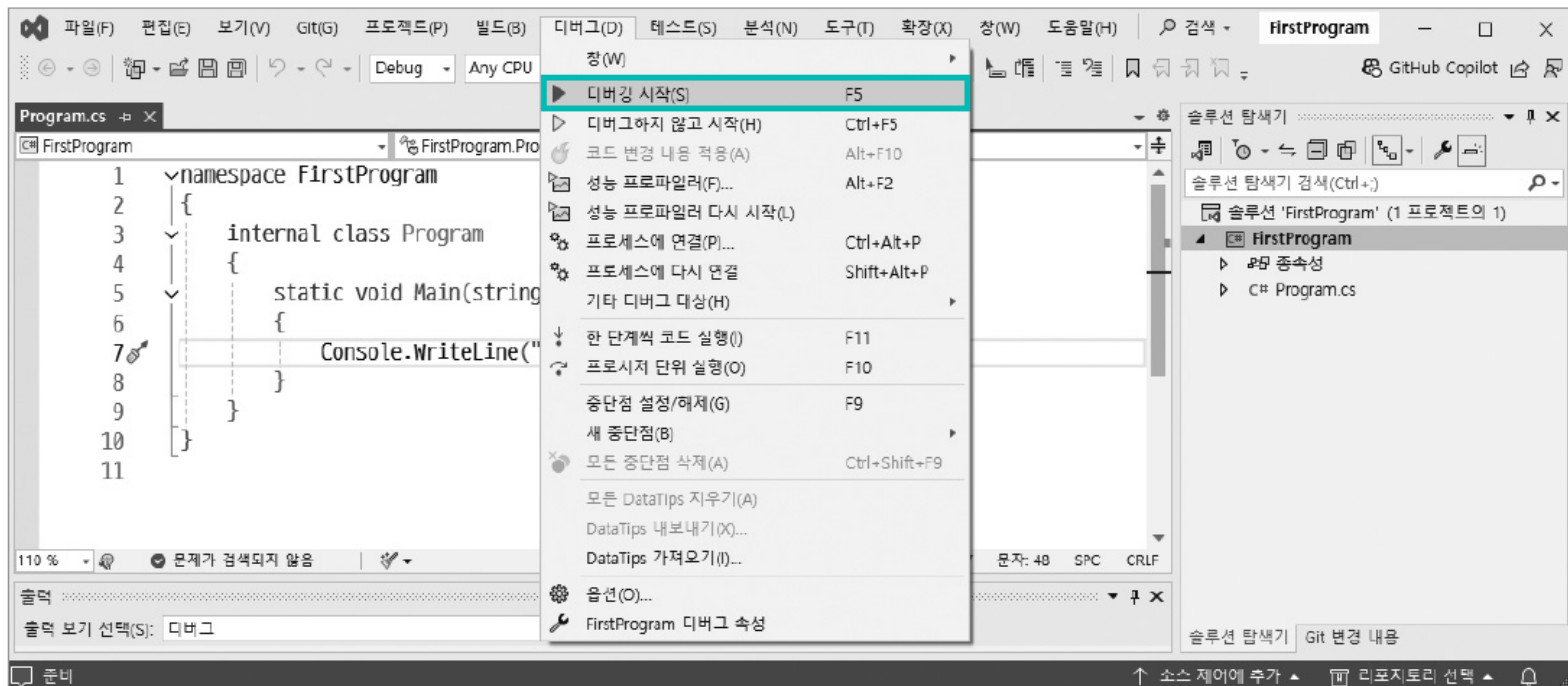
```
namespace FirstProgram
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

실습 환경 구축



프로그램 생성

- 프로젝트 실행
 - 프로젝트를 실행할 때는 상단의 [▶ 시작] 버튼 또는 [디버그]-[디버깅 시작].
 - [F5] 키도 사용 가능





프로그램 생성

- 프로젝트 실행
 - 실행하면 콘솔창이 새로 열리고 다음과 같이 출력됨

```
Hello World!
```

```
C:\Users\arin\source\repos\FirstProgram\FirstProgram\bin\Debug\net8.0\FirstProgram.exe(프로세스 21216개)이(가) 종료되었 습니다(코드: 0개).
```

```
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
```

```
이 창을 닫으려면 아무 키나 누르세요...
```

실습 환경 구축

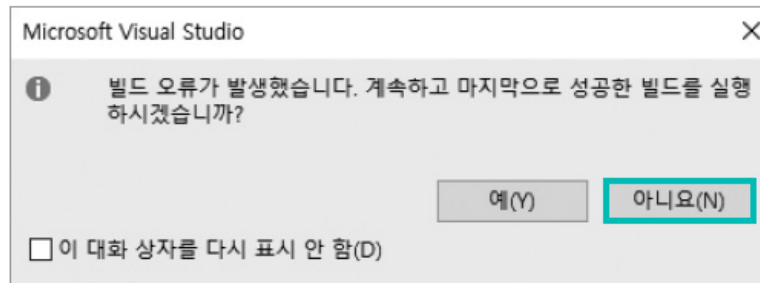


프로그램 생성

- 오류 확인 방법
 - 다음 코드를 실행

```
static void Main(string[] args)
{
    Console.WriteLine("Hello World");
}
```

- 오류가 있는 코드를 실행하면 다음과 같은 새로운 창이 나옴
- 오류가 발생했으므로 이전에 성공했던 코드로 실행하냐는 질문임. “아니요” 클릭
 - 이때 오류 창을 보면 해당 오류가 뜸. 어떤 오류인지 설명 나오며, 오류를 더블 클릭하면 해당 줄로 이동



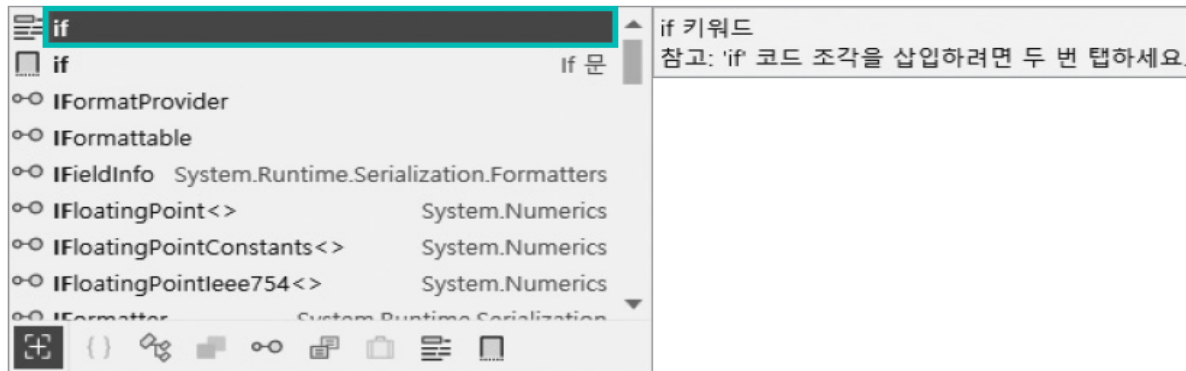
실습 환경 구축



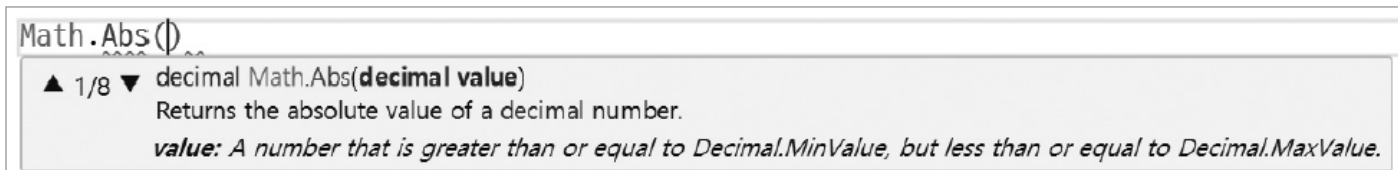
프로그램 생성

- 자동 완성 기능과 보조 기능

- 코드를 입력할 때에 [Ctrl] + [Space] 단축키를 누르면 자동 완성 기능이 실행되고 현재 위치에서 사용할 수 있는 코드가 뜬



- 메서드를 사용할 때는 해당 메서드와 관련된 설명이 뜬



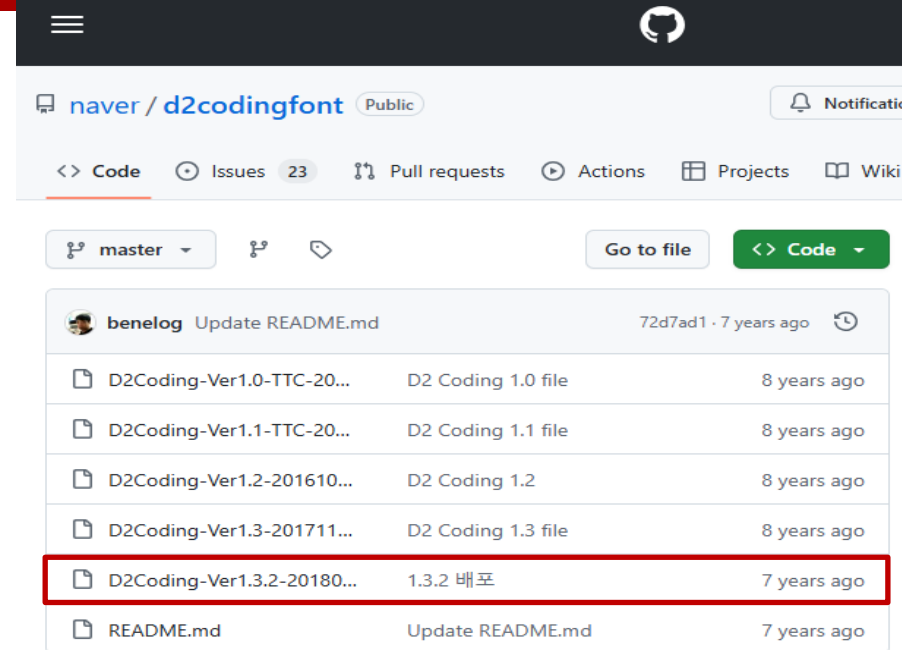
실습 환경 구축



코딩 글꼴 설치 및 등록

- D2Coding 글꼴(Ver1.3.2) 설치
 - <https://github.com/naver/d2codingfont>
 - 기본 글꼴에 없는 한글 글꼴을 추가
 - 가장 최신 버전 ZIP 파일을 선택하고
 - 하단 또는 다음 페이지에서 <Download> 버튼
 - 압축을 풀고
 - D2Coding 폴더에 있는 파일을 실행하고
 - <설치> 버튼
 - 특징

구분	폰트명	숫자	영문자	한글	특수기호
1자 형태	굴림체	1			
	Consolas	1	1		
	D2 Coding	1	1		
0자 형태	굴림체	0	0	○	
	Consolas	0	0	○	
	D2 Coding	0	0	○	
코드 기호	굴림체	() { } [] " ' " ; , . . - - + + = = _ _ /			
	Consolas	() { } [] " ' " ; , . . - - + + = = _ _ /			
	D2 Coding	() { } [] " ' " ; , . . - - + + = = _ _ /			



D2 Coding 글꼴

downloads 571k



다운로드

- [Ver 1.3.2 \(2018.06.01 배포\)](#)
- 기존 버전은 반드시 삭제후 설치 바랍니다.

실습 환경 구축



코딩 글꼴 설치 및 등록

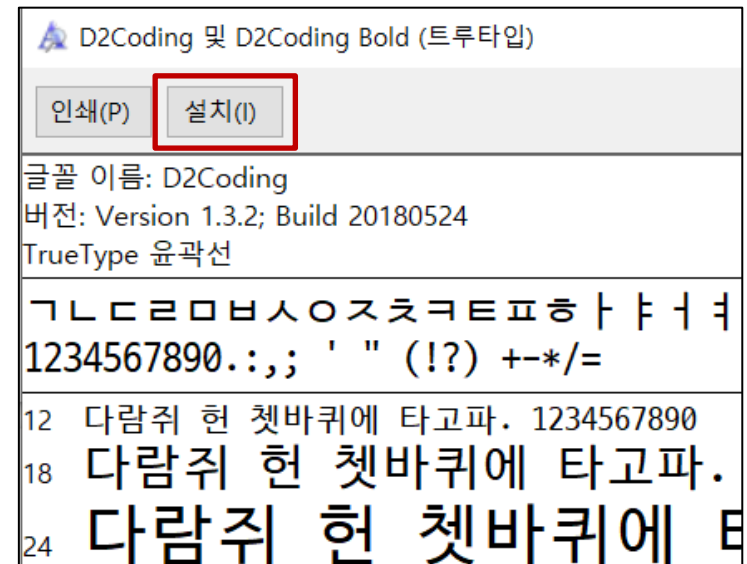
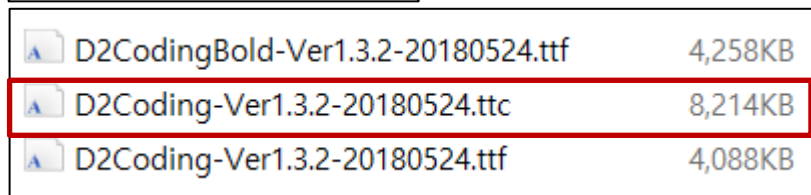
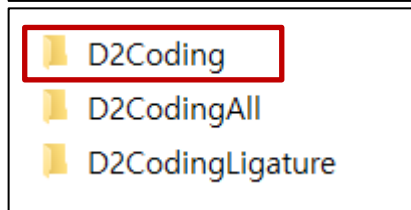
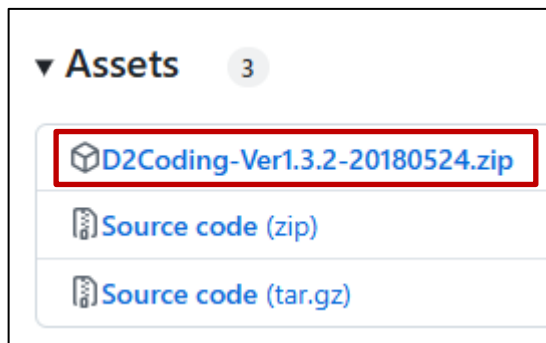
- D2Coding 글꼴(Ver1.3.2) 설치
 - <https://github.com/naver/d2codingfont>

The screenshot shows the GitHub repository page for `naver/d2codingfont`. The repository is public and has 213 forks and 2k stars. The navigation bar includes links for Code, Issues (23), Pull requests, Actions, Projects, Wiki, Security, and Insights. The current view is the Releases page, specifically for version VER1.3.2. The release title is **D2Coding Ver 1.3.2**, marked as the latest version. It was released by `benelog` on June 1, 2018, with 2 commits to master since the previous release. The release description states: "D2Coding 1.3.2 버전을 릴리즈 합니다. ligature 관련 이슈를 수정하여, ligature 적용/미적용 폰트를 구분하여 배포 합니다. 기존 버전은 반드시 삭제 후 설치 바랍니다." (We are releasing D2Coding 1.3.2. We have fixed issues related to ligatures, and we are distributing fonts with/without ligatures. Please delete the previous version and install it.) Below the description, there is a section titled **개선 내용** (Improvements).



코딩 글꼴 설치 및 등록

- D2Coding 글꼴(Ver1.3.2) 설치
 - <https://github.com/naver/d2codingfont>





코딩 글꼴 설치 및 등록

- D2Coding 글꼴(Ver1.3.2) 등록
 - VS의 [도구]-[옵션] 메뉴 선택
 - [환경]- [글꼴 및 색] 메뉴
 - 설정화면의 '글꼴'을 "D2Coding"으로 변경 (필요 시, 글자 크기나 스타일 변경)
 - VS 재실행

실습 환경 구축



한국어 보조 기능 설치

- 2020년 12월을 기준으로 Visual Studio는 보조 기능을 영어로만 출력
- 한국어로 보조 기능을 살펴보려면 한국어 팩을 다운로드하고 적용
- <https://dotnet.microsoft.com/download/intellisense>에서 5.0.x 오른쪽에 있는 '한국어' 링크를 눌러서 한국어 팩을 다운로드
 - * Net 인텔리센스 지역화 팩 다운로드 페이지

.NET Release	Localized IntelliSense Download for Windows
5.0.x	• Deutsch • español • français • italiano • 日本語 • 한국어 • português (Brasil) • русский • 中文(简体) • 中文(繁體)



한국어 보조 기능 설치

- 다운로드한 파일의 압축을 해제하면 다음과 같은 폴더가 있음
 - Microsoft.NETCore.App.Ref\ko
 - Microsoft.WindowsDesktop.App.Ref\ko
 - NETStandard.Library.Ref\ko
- 각각의 폴더를 다음 위치에 옮김 (폴더 전체 복사하기)
 - C:\Program Files\dotnet\packs\Microsoft.NETCore.App.Ref\8.0.25\ref\net8.0\
 - C:\Program Files\dotnet\packs\Microsoft.WindowsDesktop.App.Ref\8.0.25 \ref\net8.0\
 - C:\Program Files\dotnet\packs\NETStandard.Library.Ref\2.1.0\ref\netstandard2.1\

실습 환경 구축



한국어 보조 기능 설치

- 'ko'라는 이름의 폴더 전체를 복사해 넣는 것임을 주의

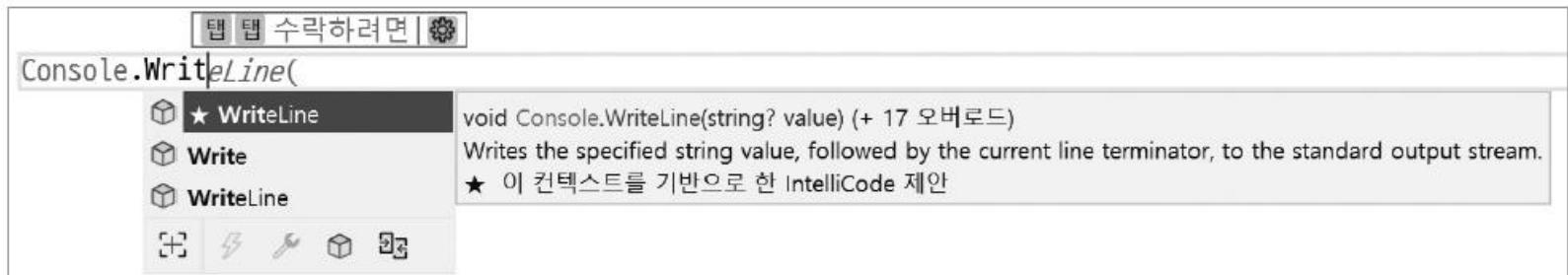




프로그램 생성

- 프로그래밍을 잘 하는 습관

- 프로그래밍 언어도 하나의 언어이기 때문에 말을 많이 해보는 것이 중요
- 자동 완성 기능을 활용해보기. Console.WriteLine 을 입력하면 다음과 같이 자동 완성됨



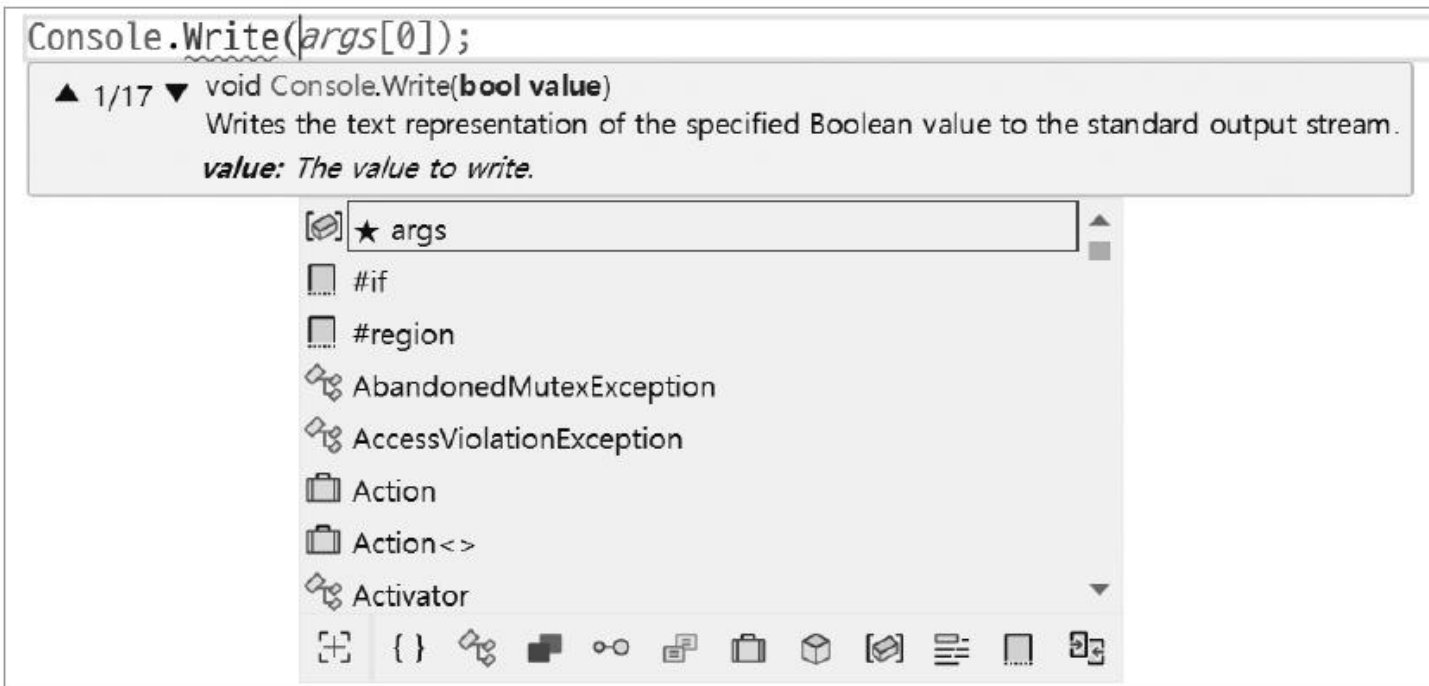
실습 환경 구축



프로그램 생성

- 프로그래밍을 잘 하는 습관

■ 기본예제 1-1 Write()와 WriteLine() 메서드 사용해보기





프로그램 생성

- 프로그래밍을 잘 하는 습관

■ 기본예제 1-1 Write()와 WriteLine() 메서드 사용해보기

코드 1-1

Write() 메서드

/1장/Write

```
01 static void Main(string[] args)
02 {
03     Console.Write("Write");
04     Console.Write("Write");
05     Console.Write("Write");
06     Console.Write("Write");
07     Console.Write("Write");
08 }
```

실행 결과

WriteWriteWriteWriteWrite



프로그램 생성

- 프로그래밍을 잘 하는 습관

■ 기본예제 1-1 Write()와 WriteLine() 메서드 사용해보기

코드 1-2

WriteLine() 메서드

/1장/WriteLine

```
01 static void Main(string[] args)
02 {
03     Console.WriteLine("WriteLine");
04     Console.WriteLine("WriteLine");
05     Console.WriteLine("WriteLine");
06     Console.WriteLine("WriteLine");
07     Console.WriteLine("WriteLine");
08 }
```

실행 결과

```
WriteLine
WriteLine
WriteLine
WriteLine
WriteLine
```



프로그램 생성

- 프로그래밍을 잘 하는 습관

■ 기본예제 1-1 Write()와 WriteLine() 메서드 사용해보기

- 자동 완성 기능을 활용을 자주 사용
- 직접 Write()와 WriteLine() 메서드의 차이를 비교해 볼 것

코드 1-3

Write()와 WriteLine() 메서드의 차이

/1장/WriteAndWriteLine

```
01 static void Main(string[] args)
02 {
03     Console.Write("Write");
04     Console.WriteLine("WriteLine");
05
06     Console.WriteLine("WriteLine");
07     Console.WriteLine("WriteLine");
08
09     Console.Write("Write");
10     Console.Write("Write");
11 }
```

실행 결과

```
WriteWriteLine
WriteLine
WriteLine
WriteWrite
```

V. 실습 및 과제

과제



자율과제

- 세부 과제
 - (교재 p.38~52) 4. 실습 환경 구축
 - C#을 공부하기 위한 Visual Studio 환경 설치를 각자 자유롭게 해봅시다.
 - 과제 제출을 위한 것이 아니며 앞으로 우리 수업의 실습을 위한 것입니다.
- 제출 시 주의사항
 - 과제 제출 없으며 각자 복습차원에서 이해하기



감사합니다

mnshim@sungkyul.ac.kr

